

HW5

A. The first one is extend thread. This way creates a class that directly extends thread. We override the run() method, and call start() to actually create a new thread.

```
class MyThread extends Thread { 2 usages
    public void run() {
        System.out.println("Thread is running");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

The second one is implement Runnable. This is more flexible because Java only allows single inheritance. The task is separated from the actual Thread object.

```
class MyRunnable implements Runnable { 1 usage
    public void run() {
        System.out.println("Runnable thread is running");
    }
}

public class Main { no usages
    public static void main(String[] args) { no usages
        Thread t = new Thread(new MyRunnable());
        t.start();
    }
}
```

The third one is implement Callable. Callable is like Runnable, but it can return a result and throw exceptions. We usually run it through a thread pool.

```
import java.util.concurrent.*;

class MyCallable implements Callable<String> { 1 usage
    public String call() { no usages
        return "Callable result";
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newSingleThreadExecutor();
        Future<String> future = executor.submit(new MyCallable());

        System.out.println(future.get());
        executor.shutdown();
    }
}
```

The last one is using Lambda. This is the shortest way to create a thread. It is commonly used when the logic is simple.

```

public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(() -> {
            System.out.println("Thread is running");
        });

        t.start();
    }
}

```

B. A thread usually moves through states like NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. For example, after calling start(), a thread goes from NEW to RUNNABLE and after finishing run(), it becomes TERMINATED. A thread can enter WAITING by calling wait() or join(), and it can go back to RUNNABLE after being notified or after another thread finishes. It may enter BLOCKED when waiting for a lock.

C. A thread pool keeps a group of reusable worker threads. When tasks come in, the pool gives them to available threads instead of creating a new thread every time. If all threads are busy, tasks usually wait in a queue. This improves performance because creating and destroying threads is expensive.

D. newCachedThreadPool can create too many threads because its maximum thread count is almost unlimited. If too many tasks arrive, it may cause high memory usage or even crash the application. newFixedThreadPool has a fixed number of threads, but its task queue is unbounded. If tasks are submitted faster than they are processed, the queue can keep growing and cause memory problems.

E. Future represents the result of an asynchronous task. It lets us check whether the task is done, cancel it, or get the result later. The problem is that future.get() is blocking. That means the current thread has to wait until the result is ready.

F. CompletableFuture is an improved version of Future. It supports non-blocking callbacks and allows us to chain multiple async tasks together.

G. Future is simple but limited. It can only get the result, cancel the task, or check status, and get() blocks the current thread. CompletableFuture is more powerful because it supports callbacks, chaining, combining tasks, and exception handling. In real projects, CompletableFuture is usually better for complex async logic.

H. synchronized is a built-in Java keyword. It is simple to use, and the lock is automatically released when the synchronized block finishes. Lock is more flexible because we can manually lock and unlock, try to get a lock, or interrupt waiting. But we must remember to release the lock in a finally block.

I. wait() makes the current thread release the lock and wait. notify() wakes up one waiting thread, while notifyAll() wakes up all waiting threads. join() means one thread waits for another thread to finish. For example, if the main thread calls t.join(), it will wait until thread t is done.

J. 1. runAsync() and supplyAsync()

runAsync() is used when the task does not return anything. supplyAsync() is used when the task returns a value.

```
CompletableFuture<Void> f1 = CompletableFuture.runAsync(() -> {
    System.out.println("No return value");
});

CompletableFuture<String> f2 = CompletableFuture.supplyAsync(() -> {
    return "Hi";
});
```

2. thenApply() and thenApplyAsync()

thenApply() transforms the previous result. thenApplyAsync() does the transformation asynchronously, usually in another thread.

```
CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> "hi")
    .thenApply(s -> s.toUpperCase());

System.out.println(future.join());

CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> "hi")
    .thenApplyAsync(s -> s.toUpperCase());

System.out.println(future.join());
```

3. handle() and exceptionally()

exceptionally() is used to provide a backup result when an exception happens. It only handles errors. handle() can handle both success and failure. It always runs.

```

CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> {
    return 10 / 0;
}).exceptionally(e -> {
    return 0;
});

System.out.println(future.join());

CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> {
    return 10 / 0;
}).handle((result, error) -> {
    if (error != null) {
        return 0;
    }
    return result;
});

System.out.println(future.join());

```

Import class... java.util.concurrent.CompletableFuture? ↵

4. thenCompose() and thenCombine()

thenCompose() is used when the next async task depends on the previous result. It avoids nested CompletableFuture. thenCombine() combines results from two independent async tasks.

```

CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> "User ID")
    .thenCompose(id -> CompletableFuture.supplyAsync(() -> id + " -> User Info"));

System.out.println(future.join());

CompletableFuture<Integer> f1 = CompletableFuture.supplyAsync(() -> 10);
CompletableFuture<Integer> f2 = CompletableFuture.supplyAsync(() -> 20);

CompletableFuture<Integer> result = f1.thenCombine(f2, (a, b) -> a + b);

System.out.println(result.join());

```

5. allOf() and anyOf()

allOf() waits for all tasks to finish. It is useful when we need every result. anyOf() returns when the first task finishes. It is useful when we only need the fastest result.

```

CompletableFuture<String> f1 = CompletableFuture.supplyAsync(() -> "A");
CompletableFuture<String> f2 = CompletableFuture.supplyAsync(() -> "B");

CompletableFuture<Void> all = CompletableFuture.allOf(f1, f2);

all.join();

System.out.println(f1.join());
System.out.println(f2.join());

CompletableFuture<String> f1 = CompletableFuture.supplyAsync(() -> "A");
CompletableFuture<String> f2 = CompletableFuture.supplyAsync(() -> "B");

CompletableFuture<Object> any = CompletableFuture.anyOf(f1, f2);

System.out.println(any.join());

```

